

Implementation Plan for an Intelligent Natural-Language CCTV Post-Analysis System Using LLMs and CV

Executive summary

You can build an accurate, auditable natural-language query system over your CCTV post-analysis data by treating LLMs as **planners and narrators** while delegating **truth** to verifiable tools: SQL over structured event tables, retrieval over vector/sparse indexes, and targeted CV re-checks on a small number of clips. This approach directly addresses known limitations of purely parametric generation—limited precision over external knowledge, weak provenance, and update difficulty—by grounding answers in a retrievable/executable store (RAG + tools). ¹

A practical build sequence is: (a) **SQL-first** NL→SQL→execution→summary (highest auditability), then (b) add **hybrid retrieval** (dense + sparse) for fuzzy questions and synonymy, then (c) introduce **agentic multi-tool orchestration** for investigative workflows, then (d) add **GraphRAG/knowledge-graph** for cross-camera movement/pattern questions at scale, and (e) expand **multimodal grounding** via CLIP-style embeddings and BLIP-2-style image-to-text verification for evidence clips. ²

To support both **real-time** and **historical** modes, use an event-stream backbone (Kafka) for ingestion and incremental processing plus a time-series/analytical store for fast time-window queries (TimescaleDB hypertables + continuous aggregates are a common fit for time-indexed event analytics). ³

Security and governance are not optional in surveillance analytics: threat models like prompt injection and insecure output handling must be mitigated with strict tool schemas, least-privilege data access, output validation, and audit logging; risk management frameworks like NIST AI RMF provide a structured lens for trustworthiness, evaluation, and lifecycle risk controls. ⁴

Assumptions and target requirements

This plan assumes these details are open-ended and should be parameterized:

- **Scale:** number of cameras, event rows/day, and retention (affects storage tiering, index sizes, and summarization cadence). ⁵
- **Latency targets:**
 - Real-time “operator mode”: seconds to tens of seconds depends on ingestion + pre-aggregation + retrieval + LLM/tool latency. Kafka is explicitly designed as a streaming platform for publishing, storing, and processing streams of records as they occur. ⁶

- Historical “investigation mode”: tens of seconds to minutes for deep queries with drill-down, especially when CV verification is triggered. ReAct-style tool-use patterns exist to reduce hallucination by interacting with external sources during reasoning. ⁷
- **LLM choice**: cloud LLM, on-prem LLM, or hybrid. The architectures below are provider-agnostic; when using a provider that supports JSON-schema-constrained tool calling and structured outputs (e.g., OpenAI ⁸), you can materially improve reliability of tool I/O. ⁹
- **Schema additions**: add a surrogate primary key such as `event_id` (hash(cam_id, timestamp, object_id, captured_event, location_tag)) to support stable evidence references in answers. This improves auditability and aligns with tool-grounding goals in RAG-style systems. ¹⁰
- **A clear boundary between “post-analysis” and “identity/bio” features**: If you later add person re-identification or biometrics, compliance and governance burdens rise substantially; use a risk-management framework to define acceptable use and controls early. ¹¹

Data layer blueprint and required transforms

Your schema is already close to an event log. Streaming platforms like Kafka explicitly treat data as timestamped records in topics, which fits CCTV event emission from CV pipelines. ⁶

A robust LLM query layer depends on deterministic transforms and indexes:

Canonical event table (SQL/time-series store)

Recommended transforms/enrichments (these are deterministic and should be reproducible from raw inputs): - `event_id` (surrogate): stable reference for audit trails. ¹²

- `ts_utc` + `ts_local` + `day_of_week`, `hour_bucket`: improves query speed and reduces LLM ambiguity. TimescaleDB hypertables chunk data by time intervals and are built for time-series/event workloads, so “time-first” keys matter. ¹³

- `episode_id`: group adjacent rows into “episodes” per cam_id/object_id, using a max gap (e.g., 2–10 seconds) to support narrative summarization and reduce index size. GraphRAG and hierarchical summarization patterns work better on episode-sized units than raw rows. ¹⁴

- `quality_flags`: derived booleans like `low_confidence = confidence < X`, `suspect_artifact = is_synthetic_artifact OR (confidence pattern)`. This supports safety filtering and enables risk-aware answers. ¹¹

Rollups for fast historical queries

TimescaleDB continuous aggregates precompute and incrementally refresh aggregates, reducing recomputation cost for dashboards and trends. ¹⁵

Use continuous aggregates / materialized rollups for: - counts by cam_id/location_tag/hour - max/avg person_count by hour - percent low-confidence by sensor/time - latency percentiles (p50/p90/p99) by detector

Text rendering for sparse + dense retrieval

Create a deterministic “event narrative string” per row and per episode. Example row string (generated, not stored by LLM):

```
2026-02-19T21:14:03Z    cam=CAM12    loc=LOBBY    event=PERSON_DETECTED
object=trk_883    people=3    conf=0.82    latency_ms=41    track_age_s=12
synthetic=false
```

RAG systems are explicitly designed to ground generation in retrievable external evidence, so the representation you index is critical. ¹⁶

Embedding strategy and storage plan

You asked for concrete model choices, dimensionality, and storage. A pragmatic plan is:

- **Text embeddings (for event narratives & episode summaries)**

- Option A (hybrid-ready, recommended): BGE-M3 dense embeddings (1024 dims) combined with sparse vectors from the same model family, enabling a hybrid retrieval stack with one embedding pipeline; BGE-M3 is commonly described as multi-function (dense + sparse) and has an embedding dimension of 1024 in published model cards. ¹⁷
- Option B (strong multilingual text retrieval): multilingual-e5-large has an embedding size of 1024 per the model card, useful if your operators query in multiple languages. ¹⁸

- **Vision embeddings (for images/clips referenced by embedding_path)**

- If your current embedding_path is CLIP-like, the common projection dimension used in popular CLIP configurations is 512 (projection_dim), and image/text are projected to the same latent dimensionality; this is explicitly documented in CLIPConfig-style references. ¹⁹
- Keep in mind CLIP's OpenAI reference implementation treats `embed_dim` as a model parameter loaded from checkpoint; so the dimension can vary by checkpoint family, and your system should store `embedding_dim` as metadata with each embedding version. ²⁰

- **Vector storage**

- For managed vector DB behavior (metadata filtering, hybrid search), Milvus documents filtered search and hybrid search patterns (vector + metadata conditions) and also supports multi-vector search for multimodal (e.g., text + image vectors). ²¹
- For local/offline indexing or edge deployments, Faiss is a well-known similarity-search library; it and related work discuss scaling similarity search and performance trade-offs, including GPU acceleration and the need for compression at high scale. ²²

Sparse indexing

Dense-only retrieval can miss exact identifiers (camera IDs, location tags, object IDs). Hybrid search combines lexical/sparse retrieval with dense semantic retrieval; this is described in official materials for both Elasticsearch-style stacks and OpenSearch. ²³

Implementation options: - separate sparse index (OpenSearch/Elasticsearch) + separate vector DB (Milvus) + fusion (RRF/weighted). ²⁴

- unified hybrid in Milvus when using a model that outputs both dense and sparse vectors (e.g., BGE-M3 tutorial). ²⁵

Reference architectures and actionable build details

The six architectures below share a common “LLM roles” pattern:

- **Router**: classify intent and pick pipeline (SQL vs retrieval vs graph vs multimodal).
- **Planner / tool caller**: produce a constrained plan (filters, time range, tools to call). ReAct motivates interleaving reasoning and action/tool calls to improve grounded performance. ⁷
- **Generator**: produce SQL or retrieval queries in structured form.
- **Summarizer**: narrate results with timestamps and evidence references (event_id / episode_id). RAG's design goal is grounded generation with retrievable evidence. ¹⁶

Visual reference architecture image

Global system flow shared by all architectures

This diagram is the “spine” you can implement once; each architecture swaps the retrieval/execution stage.

```
flowchart TD
    U[User NL query] --> R[LLM Router]
    R -->|SQL intent| S1[LLM SQL/Plan Generator]
    R -->|Semantic intent| S2[Retriever Planner]
    R -->|Graph intent| S3[Graph Planner]
    R -->|Multimodal intent| S4[Multimodal Planner]

    S1 --> DB[(Event Store: TimescaleDB/Warehouse)]
    DB --> RES1[SQL Results]

    S2 --> VDB[(Vector DB: Milvus/Faiss)]
    S2 --> SIDX[(Sparse Index: OpenSearch/Elasticsearch)]
    VDB --> RES2[Retrieved Events/Episodes]
    SIDX --> RES2

    S3 --> G[(Knowledge Graph Store)]
    G --> RES3[Subgraph + Summaries]

    S4 --> CLIPS[(Clip Store / embedding_path)]
    S4 --> CV[CV Tools: recount, re-detect, caption]
    CLIPS --> RES4[Evidence clips/frames]
    CV --> RES4

    RES1 --> SUM[LLM Summarizer]
    RES2 --> SUM
    RES3 --> SUM
    RES4 --> SUM
```

```
SUM --> OUT[Answer + Evidence + Timestamps]
OUT --> AUD[(Audit Log)]
```

The tooling focus is deliberate: tool calling is defined as a multi-step flow where the application defines tools, the model calls them, the app executes, and the model then synthesizes from tool outputs. ²⁶

Architecture comparison table

This table is qualitative/relative and should be validated with your own benchmarks; it is grounded in typical strengths described by the underlying methods (RAG, constrained NL→SQL, agentic tool use, GraphRAG, multimodal retrieval). ²⁷

Architecture	Best for	Accuracy (ground truth)	Latency	Cost	Engineering complexity	Auditability
Text-to-SQL + execution	Metrics, counts, trends, QA over strict schema	High	Low–Med	Low–Med	Med	Very High
RAG (dense + sparse) over event narratives/episodes	Fuzzy questions, story-like summaries	Med–High	Med	Med	Med	High (if citations enforced)
Agentic tool-calling (planner-executor)	Investigations spanning SQL + retrieval + CV	High (if verified)	Med–High	High	High	High
GraphRAG / knowledge-graph	Cross-entity patterns, movement pathways, “why” queries	Med–High	Med–High	Med–High	High	High
Hierarchical summarization (rollups + digests)	Long windows, executive reporting	Med	Low–Med	Low–Med	Med	Med–High
Multimodal grounding w/ clip verification	“Show me examples”, reduce CV false positives	High (on verified subset)	High	High	High	Very High

Text-to-SQL architecture

Core idea: Use the LLM to map NL → SQL (or a structured query plan compiled to SQL), execute against your event table and rollups, then narrate results with timestamps. This is the highest-auditability

foundation and should be the first production milestone. Text-to-SQL is a mature research area; Spider characterizes the challenge of generalizing to complex SQL across unseen schemas, and work like RAT-SQL improves accuracy via schema encoding/linking. ²⁸

Why it fits your schema: Most surveillance “insight” questions over your fields are aggregates (counts, peaks, correlations) and time-window filters (timestamp between ..., cam_id in ..., location_tag=...). TimescaleDB specifically targets time-series/event analytics and supports time-partitioning concepts like hypertables and incremental aggregates. ²⁹

Required transforms/enrichments - event_id primary key (for evidence). ¹²
- rollup tables/continuous aggregates for common queries (per-hour per-camera stats). ¹⁵

Indexing strategy - B-tree / BRIN indexes on (timestamp), plus composite indexes on (cam_id, timestamp) and (location_tag, timestamp). (Implementation detail; index choice depends on DB engine and data distribution—validate in your DB.)
- Continuous aggregates make “last 7 days by hour” queries fast by precomputing and incrementally refreshing. ¹⁵

LLM roles - Router → SQL Generator → SQL Executor → Summarizer.
- Use **constrained decoding / validity checks** to avoid invalid SQL generation. PICARD constrains autoregressive decoding via incremental parsing and improves text-to-SQL validity/performance. ³⁰

Prompt template (SQL generator)

(Provide schema + allowed operations + safety constraints; keep it short and deterministic.)

```
SYSTEM: You are a query compiler. Produce ONLY a JSON object matching the schema
QueryPlan.
Rules:
- Use only these fields: cam_id, captured_event, timestamp, location_tag,
  object_id,
  person_count, confidence, detector_latency_ms, source_track_age_s,
  is_synthetic_artifact.
- All queries must constrain timestamp with a start and end (UTC).
- Only SELECT queries; no writes.
- Prefer rollup tables if time_range > 24h.

USER: {natural language question}
SCHEMA (tables):
- events(event_id, cam_id, captured_event, timestamp, location_tag, object_id,
  person_count, confidence, detector_latency_ms, source_track_age_s,
  is_synthetic_artifact)
- events_hourly_rollup(cam_id, location_tag, hour_bucket, event_counts,
  avg_person_count, p95_latency_ms, ...)
```

Structured-output schema (JSON Schema draft 2020-12) JSON Schema is a standard vocabulary for describing/validating JSON structure; current versions like 2020-12 are published on json-schema.org. ³¹

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "QueryPlan",
  "type": "object",
  "required": ["intent", "time_range", "sql", "result_constraints"],
  "properties": {
    "intent": { "type": "string", "enum": ["sql_aggregate", "sql_lookup"] },
    "time_range": {
      "type": "object",
      "required": ["start_utc", "end_utc"],
      "properties": {
        "start_utc": { "type": "string", "format": "date-time" },
        "end_utc": { "type": "string", "format": "date-time" }
      }
    },
    "sql": { "type": "string" },
    "result_constraints": {
      "type": "object",
      "required": ["max_rows"],
      "properties": { "max_rows": { "type": "integer", "minimum": 1, "maximum":
5000 } }
    }
  }
}
```

Example query + expected structured output User: "Which cameras had the highest detector latency yesterday between 6pm and 11pm, and how many events did they produce?"

```
{
  "intent": "sql_aggregate",
  "time_range": {
    "start_utc": "2026-02-18T12:30:00Z",
    "end_utc": "2026-02-18T17:30:00Z"
  },
  "sql": "SELECT cam_id, COUNT(*) AS event_count, PERCENTILE_CONT(0.95) WITHIN
GROUP (ORDER BY detector_latency_ms) AS p95_latency_ms FROM events WHERE
timestamp >= '2026-02-18T12:30:00Z' AND timestamp < '2026-02-18T17:30:00Z' GROUP
BY cam_id ORDER BY p95_latency_ms DESC LIMIT 10;",
  "result_constraints": { "max_rows": 200 }
}
```

Pros/cons - Pros: strongest truthfulness/auditability; lowest hallucination risk when the LLM only compiles and the DB answers. ³²

- Cons: limited on fuzzy semantic questions unless you add a semantic layer; schema drift requires careful handling (Spider highlights generalization difficulty). ³³

RAG (dense + sparse) over event narratives and episode summaries

Core idea: Convert events/episodes to text, index with dense embeddings + sparse retrieval, retrieve top-k evidence, then generate grounded answers. RAG was introduced explicitly to combine parametric memory with non-parametric retrieval and improve factuality/specificity by conditioning on retrieved texts. ¹⁶

Required transforms/enrichments - `episode_id` + episode text ("what happened in this 30-120s window"). GraphRAG and hierarchical methods show the value of structuring corpora into higher-level summaries. ¹⁴

- "narrative string" per event row (deterministic rendering). ³⁴

Indexing strategy - Dense vectors in Milvus with scalar filters on `cam_id/location_tag/timestamp/confidence`. Milvus documents filtered search and hybrid search concepts. ³⁵

- Sparse retrieval via: - OpenSearch neural sparse + dense hybrid queries (official docs describe combining sparse and dense in hybrid queries). ³⁶

- or Elasticsearch hybrid retrieval approaches (official Elastic resources describe combining lexical and semantic retrieval). ³⁷

Embedding strategy - Use BGE-M3 for text embeddings (1024 dims) and optionally its sparse vectors; Milvus provides a hybrid-search tutorial using BGE-M3 and states it can convert text to dense and sparse vectors. ³⁸

- Store embedding version: `{model_name, model_version, dim, normalization}` to support reindexing and A/B tests. Vector retrieval systems often require benchmarking and trade-off tuning; Faiss resources emphasize accuracy/speed/memory trade-offs. ³⁹

Retrieval pipeline 1) Router chooses `rag_semantic`. ³⁴

2) Retrieve candidate episodes via dense + sparse hybrid. ⁴⁰

3) Rerank (optional) using a cross-encoder or LLM-scoring, then pass top evidence snippets + metadata to summarizer.

4) Summarizer emits answer and cites evidence IDs.

Prompt template (RAG summarizer)

SYSTEM: You are a surveillance analyst assistant. Answer ONLY using the evidence blocks.

If evidence is insufficient, say what is missing and propose a narrower query.

USER QUESTION: {q}

EVIDENCE BLOCKS:

- [episode_id=E123, cam_id=..., start=..., end=..., summary=...]
- [event_id=..., timestamp=..., captured_event=..., person_count=..., confidence=...]

Structured output (answer object)

The objective is to force “time-stamped, evidence-backed” outputs and mitigate insecure output handling risks highlighted by OWASP. ⁴¹

```
{
  "answer_text": "...",
  "time_bounds": {"start_utc": "...", "end_utc": "..."},
  "key_findings": [
    {"finding": "...", "supporting_event_ids": ["..."], "confidence": 0.0}
  ],
  "evidence": [
    {"event_id": "...", "timestamp": "...", "cam_id": "...", "location_tag": "..."}
  ],
  "caveats": ["..."]
}
```

Pros/cons - Pros: handles synonyms and “story” questions; scalable via retrieval; grounded generation is the point of RAG. ¹⁶

- Cons: retrieval errors become answer errors; you must measure retrieval recall/precision and enforce citation discipline. ⁴²

Agentic tool-calling (planner-executor) for investigative workflows

Core idea: The LLM produces an explicit plan, calls tools (SQL, vector search, CV verification), then synthesizes. ReAct formalizes a reasoning+acting paradigm where models interleave reasoning traces with actions and use external interactions to reduce hallucination/error propagation. ⁴³

Required transforms/enrichments - Tool contract definitions (schemas) and strict validation. JSON Schema provides the vocabulary; structured outputs that guarantee adherence to schema can reduce tool-call brittleness. ⁴⁴

- “CV verification tools” need deterministic outputs: recount person_count, re-run detector for a clip, summarize clip. Video/vision-language methods like BLIP-2 exist to bridge frozen image encoders and LLMs for image-to-text generation. ⁴⁵

Index strategy - same as SQL + RAG, but tool planner decides which index first.

Verification loop with CV tools - When confidence is low or is_synthetic_artifact is true, the planner should:

- 1) retrieve the corresponding clip/frame via embedding_path,
- 2) run a CV re-check (detector + counter),
- 3) optionally run a captioning/VLM summarizer,
- 4) reconcile outputs into “final statement + confidence”. ⁴⁶

Plan schema (example) This is where tool reliability matters most; the more constrained this schema, the safer and more auditable the system. ⁴⁷

```

{
  "intent": "investigation",
  "time_range": {"start_utc": "...", "end_utc": "..."},
  "steps": [
    {"tool": "sql_query", "args": {"sql": "...", "max_rows": 500}},
    {"tool": "vector_search", "args": {"query": "...", "top_k": 50, "filters":
{"location_tag": "LOBBY"}}},
    {"tool": "cv_verify_person_count", "args": {"embedding_path": "...",
"method": "recount"}}
  ],
  "final_answer_format": "AnswerWithEvidence"
}

```

Mermaid dataflow (agentic)

```

sequenceDiagram
    participant Analyst as User
    participant Router as LLM Router
    participant Planner as LLM Planner
    participant SQL as SQL Engine
    participant VDB as Vector Search
    participant CV as CV Verify
    participant Summ as LLM Summarizer

    Analyst->>Router: NL query
    Router->>Planner: Choose investigation path
    Planner->>SQL: Execute SQL (time bounds + filters)
    SQL-->>Planner: aggregates + candidate events
    Planner->>VDB: Retrieve similar episodes/events
    VDB-->>Planner: evidence set
    Planner->>CV: Verify subset (low confidence / high impact)
    CV-->>Planner: verified counts/captions
    Planner->>Summ: Provide evidence + tool outputs
    Summ-->>Analyst: Answer + timestamps + evidence IDs

```

Latency/cost tradeoffs - Higher latency than SQL-only or RAG-only because tool calls are sequential and CV verification is expensive; however, it can deliver higher correctness if verification is scoped to a small subset. ReAct explicitly centers the value of external interactions during reasoning. 48

GraphRAG / knowledge-graph architecture

Core idea: Build a knowledge graph of entities and relationships (cam_id, location_tag, object_id, episodes, transitions), then answer questions using graph-aware retrieval and summaries. GraphRAG proposes a graph-based approach that (a) constructs a knowledge graph, (b) partitions it into communities, and (c) generates community-level summaries to improve query-focused summarization at scale. 14

Required transforms - Graph edges:

- SEEN_AT(object_id → cam_id/location_tag @ time)
- NEXT_SEEN(object_id @ cam_a → cam_b within Δt)
- CO_OCCURS(location_tag ↔ event types within window)
- Community summaries: generated offline weekly/daily; used as “global context” during queries. This mirrors GraphRAG’s published pipeline stages. ⁴⁹

Indexing - Graph store (Neo4j-like or property graph) + vector index over summaries and node/edge descriptions.

- Hybrid retrieval still helps: exact object_id queries are sparse-friendly; semantic pathway queries are dense-friendly. Hybrid retrieval is broadly described as combining lexical and semantic signals. ⁵⁰

Mermaid component diagram (GraphRAG-oriented)

```
flowchart LR
    EV["EV[Events/Episodes]"] --> EX["EX[Graph Extractor]"]
    EX --> KG["KG[Knowledge Graph]"]
    KG --> COMM["COMM[Community Detection/Partition]"]
    COMM --> SUMS["SUMS[LLM Community Summaries]"]
    SUMS --> VIDX["VIDX[(Vector Index over summaries)]"]
    Q["Q[User Query]"] --> R["R[Router]"]
    R --> GR["GR[Graph Retriever]"]
    GR --> KG
    GR --> VIDX
    KG --> EVID["EVID[Subgraph Evidence]"]
    VIDX --> CONT["CONT[Summary Context]"]
    EVID --> LLM1["LLM[LLM Answer Synthesizer]"]
    CONT --> LLM1
    LLM1 --> OUT["OUT[Answer + supporting nodes/edges + timestamps]"]
```

Pros/cons - Pros: strong for “relationship” questions (“common movement patterns”, “co-occurring anomalies”) that are hard in plain SQL. GraphRAG is designed to scale sensemaking over large corpora via graph structure and summaries. ¹⁴

- Cons: highest modeling complexity; graph quality depends on consistent entity resolution (especially object_id semantics). ⁵¹

Hierarchical summarization architecture

Core idea: Precompute multi-resolution summaries and rollups (minute→hour→day→week) so LLM queries rarely need to scan raw events. This is the most cost-efficient way to support very long time windows.

Implementation anchors - TimescaleDB continuous aggregates explicitly exist to precompute and incrementally refresh aggregate data using materialized-view concepts, reducing how much must be recomputed on read. ¹⁵

- GraphRAG's use of community-level summaries provides another precedent: build summaries offline that can be retrieved at query-time to give global context efficiently. ⁴⁹

Required transforms - Hourly and daily "digests" per cam_id/location_tag that include: peaks, anomalies, top events, percent low-confidence, percent synthetic.

- Store as documents in retrieval index (dense + sparse) and as rows in SQL rollups.

Retrieval pipeline 1) Router identifies "long window summary" intent (e.g., time range > 7 days).

2) Retrieve day/week summaries first; if the user asks "show evidence," drill down to episodes then events.

3) Emit answer with a strict "summary→drilldown" trace to remain auditable.

Pros/cons - Pros: cheapest and fastest for long periods; predictable performance. ⁵²

- Cons: summaries can hide rare events unless drill-down policy is strong; you must keep summary regeneration consistent.

Multimodal grounding and evidence verification

Core idea: Use your embedding_path to power "show me examples" and "verify this claim" workflows. CLIP aligns images and text in a joint embedding space, enabling similarity search and zero-shot concepts; BLIP-2 shows a strategy to connect vision encoders to LLMs for instruction-following image-to-text generation. ⁵³

Indexing strategy - Store image/clip vectors in Milvus; Milvus supports multi-vector and hybrid search, useful when you want text+image retrieval in one store. ⁵⁴

- Store per-clip metadata (cam_id, timestamp, location_tag, confidence) for scalar filtering. ⁵⁵

Verification loop - Use CV tools as "ground truth generators."

- Trigger verification when: - confidence < threshold, or

- is_synthetic_artifact=true, or

- query involves compliance/high-stakes claims ("was there crowding above 50?").

This aligns with risk-management principles that emphasize evaluation and reliability in deployment. ⁵⁶

Example multimodal query User: "Show me example clips of 'crowding' near the lobby last weekend, and confirm the max person count for each."

Expected structured output (high level):

```
{
  "intent": "multimodal_evidence",
  "time_range": {"start_utc": "...", "end_utc": "..."},
  "definition": {"crowding_person_count_gte": 20},
  "retrieval": {
    "vector_query": {"model": "clip", "top_k": 30},
    "filters": {"location_tag": "LOBBY", "confidence_gte": 0.6}
  },
  "verification": {
```

```

    "tools": [{"name": "cv_verify_person_count", "max_items": 10}],
    "policy": "verify_top_by_similarity_then_by_uncertainty"
  },
  "output": {
    "items": [
      {"cam_id": "...", "timestamp": "...", "embedding_path": "...",
"verified_person_count": 27}
    ]
  }
}

```

Security, governance, and safety controls

This system must be designed as a secured data-access product, not just a chatbot.

Threat model and controls

The OWASP ⁵⁷ Top 10 for LLM applications highlights risks like prompt injection and insecure output handling; the genAI risk pages spell out that prompt injection manipulates model behavior through crafted inputs, and insecure output handling is about insufficient validation/sanitization of model outputs before downstream execution. ⁵⁸

Translated into concrete controls: - Strict tool schemas + server-side validation (reject unknown fields, cap time ranges, cap result sizes). ⁵⁹

- Read-only SQL role; deny network egress from tool executor; allowlist tables/columns.
- Prompt-injection-resistant design: isolate user content, never let the model directly execute code; tool calls must be validated, logged, and replayable. ⁶⁰
- “Insecure output handling” mitigation: treat the model output as untrusted input; sanitize before passing to DB/CV tools. ⁶¹

Risk management and governance

The National Institute of Standards and Technology ⁶² AI RMF is intended for voluntary use to manage risks and incorporate trustworthiness across AI system lifecycles. Apply it by requiring: - documented intended use + prohibited use, - measured reliability targets, - monitoring for drift and incidents, - human oversight workflows for high-impact outputs. ¹¹

Auditability requirements - Every answer should store: query, plan JSON, executed SQL, retrieved evidence IDs, verification actions, model+embedding versions, and latency/cost metrics. RAG’s provenance motivation and tool-calling’s multi-step flow both support this pattern. ⁶³

Evaluation, benchmarking, and test datasets

Your evaluation must separate **(1) query interpretation accuracy** from **(2) retrieval accuracy** from **(3) summarization faithfulness**.

Text-to-SQL evaluation - Use execution accuracy and exact match where applicable; Spider is a canonical benchmark for complex cross-domain text-to-SQL and highlights schema generalization difficulty. ⁶⁴

- For conversational follow-ups (“now compare with...”, “drill down”), CoSQL provides conversational text-to-

SQL challenges and data. ⁶⁵

- If you generate SQL directly, apply constraints like PICARD to reduce invalid SQL generation; PICARD was proposed specifically because unconstrained auto-regressive decoding often generates invalid SQL. ³⁰

RAG / retrieval evaluation - Build a labeled set of internal queries (200–1000) with known relevant episodes/events. Then compute: Recall@k, MRR/NDCG, and evidence coverage. Hybrid retrieval is motivated by its ability to combine lexical precision and semantic matching; official hybrid-search explanations describe lexical+semantic fusion. ⁵⁰

- Benchmark vector index performance/recall trade-offs; Faiss resources discuss speed/accuracy/memory trade-offs and scaling issues requiring compression at very large sizes. ³⁹

Multimodal verification evaluation - Measure “verification gain”: reduction in false positives/false claims after CV re-check on a sampled subset, and the incremental latency/cost. BLIP-2 and CLIP provide anchor capabilities for vision-language mapping and image-to-text generation in modern pipelines. ⁶⁶

System-level metrics - End-to-end answer faithfulness: % of claims supported by evidence IDs.

- “Unsupported claim rate”: must trend toward zero in production for high-stakes deployments; grounding is a core motivation of RAG and tool-use paradigms. ²⁶

- Latency histograms by pipeline type (SQL-only, RAG, agentic+CV). Kafka ingestion lag and Timescale DB query latency should be tracked as part of streaming/time-series operation. ⁶⁷

Rollout, deployment options, and monitoring

Real-time mode (streaming) - CV pipelines emit events to Kafka topics partitioned by cam_id or location_tag; Kafka is designed for publish/subscribe, durable storage, and processing of streams of records. ⁶

- Stream processor writes to:

- TimescaleDB hypertable for recent hot data (e.g., 7–30 days), leveraging chunking by time, and

- Vector/sparse indexes for immediate semantic search. ⁶⁸

- Continuous aggregates provide near-real-time rollups with incremental refresh. ⁶⁹

Historical mode (batch/backfill) - Backfill raw events into cold storage + warehouse or TimescaleDB “cold hypertables” with larger chunk intervals; reindex embeddings and regenerate summaries in batch. - GraphRAG builds are naturally batch-oriented: graph construction + community summarization are offline steps used at query-time. ⁴⁹

Suggested phased rollout This timeline is intentionally incremental to minimize risk while achieving early value.

```
gantt
  title CCTV NL Query System Rollout
  dateFormat YYYY-MM-DD
  axisFormat %b %d
  section Foundation
  Canonical schema + event_id + rollups      :a1, 2026-02-20, 21d
  Kafka ingestion + Timescale hypertables    :a2, after a1, 21d
```

section Phase 1	
NL->SQL (read-only) + audited summaries	:b1, after a2, 28d
section Phase 2	
RAG index (dense+sparse) + evidence citations	:c1, after b1, 28d
section Phase 3	
Agentic investigations + CV verification tools	:d1, after c1, 35d
section Phase 4	
Graph/GraphRAG pathway analytics + summaries	:e1, after d1, 42d

The ordering is motivated by auditability: SQL-first is easiest to verify, then retrieval, then multi-tool agentic workflows, then graph-level reasoning. This aligns with research motivations: RAG improves grounded answering via retrieval; ReAct improves grounded reasoning via action; GraphRAG improves global sensemaking via graph structure and summaries. ⁷⁰

Monitoring and alerting checklist - Ingestion: Kafka lag, per-cam throughput, event drop rates. ⁷¹

- Storage: Timescale chunk growth, continuous aggregate refresh latency. ⁷²

- Retrieval: Recall@k on a replay set; vector index latency/CPU. ⁷³

- LLM layer: tool-call failure rate, invalid-plan rate (should approach zero with strict schema validation), unsupported-claim rate. OWASP's taxonomy motivates explicit monitoring for prompt injection and insecure output handling classes. ⁴¹

- Governance: periodic risk review and evaluation updates consistent with a lifecycle approach to AI risk management. ⁷⁴

¹ ⁸ ⁹ ¹⁰ ¹⁶ ²⁶ ²⁷ ³² ³⁴ ⁶³ ⁷⁰ **Function calling | OpenAI API**

https://developers.openai.com/api/docs/guides/function-calling/?utm_source=chatgpt.com

² ²⁸ ³³ ⁶⁴ **<https://arxiv.org/abs/1809.08887>**

<https://arxiv.org/abs/1809.08887>

³ ⁵ ⁶ ¹⁷ ⁶⁷ ⁷¹ **bge-m3 Model by BAAI**

https://build.nvidia.com/baai/bge-m3/modelcard?utm_source=chatgpt.com

⁴ ¹² ⁴¹ ⁴⁷ ⁵⁸ ⁶² **<https://owasp.org/www-project-top-10-for-large-language-model-applications/>**

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

⁷ ⁴³ ⁴⁸ **Structured model outputs | OpenAI API**

https://developers.openai.com/api/docs/guides/structured-outputs/?utm_source=chatgpt.com

¹¹ ⁵⁶ **<https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>**

<https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>

¹³ ⁶⁸ ⁷² **<https://www.tigerdata.com/docs/use-timescale/latest/hypertables>**

<https://www.tigerdata.com/docs/use-timescale/latest/hypertables>

¹⁴ **<https://arxiv.org/abs/2404.16130>**

<https://arxiv.org/abs/2404.16130>

¹⁵ **<https://www.tigerdata.com/docs/use-timescale/latest/continuous-aggregates>**

<https://www.tigerdata.com/docs/use-timescale/latest/continuous-aggregates>

18 [intfloat/multilingual-e5-large](#)

https://huggingface.co/intfloat/multilingual-e5-large?utm_source=chatgpt.com

19 https://huggingface.co/docs/transformers/en/model_doc/clip

https://huggingface.co/docs/transformers/en/model_doc/clip

20 <https://raw.githubusercontent.com/openai/CLIP/main/clip/model.py>

<https://raw.githubusercontent.com/openai/CLIP/main/clip/model.py>

21 35 46 55 [BAAI/bge-large-en-v1.5](#)

https://huggingface.co/BAAI/bge-large-en-v1.5?utm_source=chatgpt.com

22 39 42 73 [State-of-art retrieval-augmented LLM: bge-large-en-v1.5](#)

https://medium.com/%40marketing_novita.ai/state-of-art-retrieval-augmented-llm-bge-large-en-v1-5-4cd5abbcfb0a?utm_source=chatgpt.com

23 37 40 50 <https://www.elastic.co/what-is/hybrid-search>

<https://www.elastic.co/what-is/hybrid-search>

24 <https://www.elastic.co/search-labs/fr/blog/improving-information-retrieval-elastic-stack-hybrid>

<https://www.elastic.co/search-labs/fr/blog/improving-information-retrieval-elastic-stack-hybrid>

25 30 38 [Hybrid Search with Milvus](#)

https://milvus.io/docs/hybrid_search_with_milvus.md?utm_source=chatgpt.com

29 <https://github.com/timescale/timescaledb>

<https://github.com/timescale/timescaledb>

31 44 59 [JSON Schema - Specification \[#section\]](#)

https://json-schema.org/specification?utm_source=chatgpt.com

36 <https://docs.opensearch.org/latest/vector-search/ai-search/neural-sparse-search/>

<https://docs.opensearch.org/latest/vector-search/ai-search/neural-sparse-search/>

45 [BAAI/bge-m3](#)

https://huggingface.co/BAAI/bge-m3?utm_source=chatgpt.com

49 51 <https://arxiv.org/html/2404.16130v2>

<https://arxiv.org/html/2404.16130v2>

52 69 <https://www.tigerdata.com/docs/use-timescale/latest/continuous-aggregates/about-continuous-aggregates>

<https://www.tigerdata.com/docs/use-timescale/latest/continuous-aggregates/about-continuous-aggregates>

53 66 [Multi-Vector Hybrid Search](#)

https://milvus.io/docs/multi-vector-search.md?utm_source=chatgpt.com

54 57 [MetaKaggle Forum Data BGE-M3 Embeddings](#)

https://www.kaggle.com/datasets/bwandowando/meta-kaggle-forum-data-embeddings-with-baaibge-m3?utm_source=chatgpt.com

60 <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

61 <https://genai.owasp.org/llmrisk2023-24/llm02-insecure-output-handling/>

<https://genai.owasp.org/llmrisk2023-24/llm02-insecure-output-handling/>

⁶⁵ <https://aclanthology.org/D19-1204/>

<https://aclanthology.org/D19-1204/>

⁷⁴ <https://www.nist.gov/itl/ai-risk-management-framework>

<https://www.nist.gov/itl/ai-risk-management-framework>